



## Unit1

### Introduction to JAVA

Java is a programming language and a platform. Java is a high level, robust, object-oriented and secure programming language.

Java was developed by Sun Microsystems in the year 1995. James Gosling is known as the father of Java. Before Java, its name was Oak. Since Oak was already a registered company, so James Gosling and his team changed the name from Oak to Java.

### **History of Java**

Java is one of the world's most popular programming languages. It was developed to create software that could run on many different devices without modification. Its history spans more than three decades and has significantly influenced modern software development.

#### The Beginning (1991)

Java was created in 1991 by a team of engineers at Sun Microsystems by James Gosling. Other key members included Mike Sheridan and Patrick Naughton.

The project was initially called the Green Project. Its goal was to develop software for consumer electronic devices such as televisions, set-top boxes, and home appliances. At that time, different devices used different operating systems, making software development difficult. The team wanted a language that could run on any device.

#### The Oak Programming Language (1991–1994)

The language was first named Oak, after an oak tree outside James Gosling's office. Features of Oak included:

- Object-oriented programming
- Platform independence
- Automatic memory management
- Security features

Simplicity compared to languages like C++

However, the name "Oak" was already trademarked by another company, so it could not be used commercially

#### Renamed to Java (1995)

In 1995, Oak was renamed Java. According to the development team, the name was inspired by Java coffee, which they drank frequently during development.

Sun Microsystems officially introduced Java with the slogan:

"Write Once, Run Anywhere (WORA)."

This slogan reflected Java's greatest advantage: programs compiled into bytecode could run on any device that had a Java Virtual Machine (JVM)

### Rise of Java on the Internet

(1995–1999) The mid-1990s saw the rapid growth of the Internet.

Java became popular because:

- It supported interactive web applications.
- Java applets could run inside web browsers.
- It was secure and portable.
- It worked across different operating systems.

Although applets were eventually phased out, Java's portability helped establish it as a major programming language.

### Java 2 Platform (1998)

In 1998, Sun released Java 2, introducing three major editions:

- Java SE (Standard Edition) – Desktop applications and core programming.
- Java EE (Enterprise Edition) – Large-scale enterprise and web applications.
- Java ME (Micro Edition) – Mobile phones and

embedded devices. This made Java suitable for a wide variety of software development PGrowth in Enterprise

### Applications (2000s)

During the 2000s, Java became one of the most widely used languages for enterprise software.

### Oracle Acquires Sun Microsystems (2010)

In 2010, Oracle Corporation acquired Sun Microsystems. After the acquisition:

Oracle became responsible for Java development. Java continued to receive regular updates.

Performance, security, and language features were improved.

## JAVA and Network

Java is strongly associated with the Internet because the first application program written in Java was HotJava, a browser to run the applet on Internet. So the Internet users use the Java to create the applet programs and run them locally using a 'Java-enabled browsers' like HotJava..

Java is a platform-independent, object-oriented programming language (OOP). It is not to be confused with JavaScript, a scripting language used to create dynamic web pages. Due to its reliability and ease of use, Java is one of the most popular programming languages in the world.

### **Java Internet security :**

Java™ programming is becoming increasingly widespread in today's computing environments. You should prepare to deal with the security factors that are associated with Java.

Although a firewall is a good defense against most general Internet security risks, it does not provide protection for many risks that using Java presents. Your security policy should include details for protecting your system against three areas of concern for Java: applications, applets, and servlets. Also, you should understand how Java and resource security interact in terms of authentication and authorization for Java programs.

### **Java applications**

As a language, Java has some characteristics that protect Java programmers from unintentional errors that can cause integrity problems. (Other languages that are commonly used for PC applications, such as C or C++, do not protect the programmers from unintentional errors as strongly as Java does.) For example, Java uses strong typing, the strict enforcement of type rules with no exceptions, to protect the programmer from using objects in unintended ways. Java does not allow pointer manipulation, which protects the programmer from accidentally going outside the memory boundaries of the program. From an application development perspective, you can view Java as you do other high-level languages. You need to apply the same security rules for application design that you apply with other languages on your system.

### **Java applets**

Java applets are small Java programs that you can include in HTML pages that run on the client but have the potential to access your i5/OS operating system. An Open Database Connectivity (ODBC) program or an advanced program-to-program communications (APPC)

program that operates on a PC in your network can also potentially access your operating system when, for example, your system is being used to serve applications or is being used as a Web server. In general, Java applets can establish a session only with the i5/OS operating system from which the applet originated. Therefore, a Java applet can access your i5/OS operating system from a connected PC only when the applet comes from that i5/OS operating system.

IITM/MCA

An applet can attempt to connect to any TCP/IP port on a system. It does not need to talk to a software server that is written in Java. But, for systems that are written with the IBM® Toolbox for Java, the applet must provide a user ID and password when it establishes connections back to the system. In this material, the systems described are all i5/OS operating systems. (A Java application server does not need to use the IBM Toolbox for Java.) Typically, the IBM Toolbox for Java class prompts the user for a user ID and password for the first connection.

The applet can perform functions on the i5/OS operating system only if the user profile has authorization to those functions. Therefore, a good resource security scheme is essential when you begin to use Java applets to provide new application functions. When the system processes the requests from applets, it does not use the limited capability value that is specified in the user profile.

The applet viewer allows you to test an applet on the i5/OS operating system; however, it is not subject to browser security restrictions. Therefore, you need to use the applet viewer to test only your own applets, never to run applets from outside sources. Java applets often write to the PC drive of the user, which might provide the applet with the opportunity to perform a destructive action. However, you can use a digital certificate to sign a Java applet to establish its authenticity. The signed applet can write to the PC's local drives, even though the default setting for the browser prevents it. The signed applet can also write to mapped drives on your system because they appear to the PC to be local drives.

For Java applets that originate from your system, you might need to use signed applets. However, you need to instruct your users, in general, not to accept signed applets from unknown sources.

Beginning with V4R4, you can use the IBM Toolbox for Java to set up a Secure Sockets Layer (SSL) environment. You can also use the IBM Developer Toolkit for Java to make a Java application secure with SSL. Using SSL with your Java applications ensures encryption of the data, including the user IDs and passwords that pass between the client and server. You can use Digital Certificate Manager (DCM) to configure registered Java programs to use SSL.

## **Java servlets**

Servlets are server-side components that are written in Java, which dynamically extend the function of a Web server without changing Web server code. The IBM WebSphere® Application Server that is included in IBM Web Enablement for i5/OS provides support for using servlets on the i5/OS operating systems.

You must use resource security on servlet objects that the system uses. However, applying resource security to a servlet does not sufficiently secure it. After a Web server loads a

servlet, resource security does not prevent others from running it too. Consequently, you need to use resource security in addition to using HTTP server security controls and directives. For example, do not allow servlets to run under the profile of the Web server only. You also need to use the security features provided by your servlet development tools, such as those found in the WebSphere Application Server for i5/OS.

IBM Toolbox for Java contains security classes to provide verification of the identity of the user and optionally to assign that identity to the operating system thread for an application or servlet that is running on an i5/OS operating system. Subsequent checks for resource security occur under the assigned identity.

The IBM Developer Kit for Java provides support for the Java Authentication and Authorization Service (JAAS), which is a standard extension to the Java 2 Software Development Kit (J2SDK), Standard Edition. Currently, J2SDK provides access controls that are based on where the code originated and who signed the code (code source-based access controls).

#### Securing your Java applications with SSL

You can use Secure Sockets Layer (SSL) to secure communications for i5/OS applications that you develop with IBM Developer Kit for Java. Client applications that use IBM Toolbox for Java can also take advantage of SSL. The process for enabling SSL for your own Java applications is somewhat different from the process for enabling SSL for the other applications.

## Features of Java

The primary objective of Java programming language creation was to make it portable, simple and secure programming language. Apart from this, there are also some excellent features which play an important role in the popularity of this language. The features of Java are also known as Java buzzwords.

A list of the most important features of the Java language is given below.

### Simple

Java is very easy to learn, and its syntax is simple, clean and easy to understand. According to Sun Microsystems, Java language is a simple programming language because:

- Java syntax is based on C++ (so easier for programmers to learn it after C++).
- Java has removed many complicated and rarely-used features, for example, explicit pointers, operator overloading, etc.
- There is no need to remove unreferenced objects because there is an Automatic Garbage Collection in Java.

### **Object-oriented**

Java is an object-oriented programming language. Everything in Java is an object. Object-oriented means we organize our software as a combination of different types of objects that incorporate both data and behavior.

Object-oriented programming (OOPs) is a methodology that simplifies software development and maintenance by providing some rules.

Basic concepts of OOPs are:

1. Object
2. Class
3. Inheritance
4. Polymorphism
5. Abstraction
6. Encapsulation

### **Platform Independent**

Java is platform independent because it is different from other languages like C, C++, etc. which are compiled into platform specific machines while Java is a write once, run anywhere language. A platform is the hardware or software environment in which a program runs. There are two types of platforms software-based and hardware-based. Java provides a software-based platform.

The Java platform differs from most other platforms in the sense that it is a software-based platform that runs on top of other hardware-based platforms. It has two components:

1. Runtime Environment
2. API(Application Programming Interface)

Java code can be executed on multiple platforms, for example, Windows, Linux, Sun Solaris, Mac/OS, etc. Java code is compiled by the compiler and converted into bytecode. This bytecode is a platform-independent code because it can be run on multiple platforms, i.e., Write Once and Run Anywhere (WORA).

### **Secured**

Java is best known for its security. With Java, we can develop virus-free systems. Java is secured because:

- No explicit pointer
- Java Programs run inside a virtual machine sandbox
- Classloader: Classloader in Java is a part of the Java Runtime Environment (JRE) which is used to load Java classes into the Java Virtual Machine dynamically. It adds

security by separating the package for the classes of the local file system from those that are imported from network sources.

- Bytecode Verifier: It checks the code fragments for illegal code that can violate access rights to objects.
- Security Manager: It determines what resources a class can access such as reading and writing to the local disk.

Java language provides these securities by default. Some security can also be provided by an application developer explicitly through SSL, JAAS, Cryptography, etc.

### **Robust**

The English meaning of Robust is strong. Java is robust because:

- It uses strong memory management.
- There is a lack of pointers that avoids security problems.
- Java provides automatic garbage collection which runs on the Java Virtual Machine to get rid of objects which are not being used by a Java application anymore.
- There are exception handling and the type checking mechanism in Java. All these points make Java robust.
- 

### **Architecture-neutral**

Java is architecture neutral because there are no implementation dependent features, for example, the size of primitive types is fixed.

In C programming, int data type occupies 2 bytes of memory for 32-bit architecture and 4 bytes of memory for 64-bit architecture. However, it occupies 4 bytes of memory for both 32 and 64-bit architectures in Java.

### **Portable**

Java is portable because it facilitates you to carry the Java bytecode to any platform. It doesn't require any implementation.

### **High-performance**

Java is faster than other traditional interpreted programming languages because Java bytecode is "close" to native code. It is still a little bit slower than a compiled language (e.g., C++). Java is an interpreted language that is why it is slower than compiled languages, e.g., C, C++, etc.

### **Distributed**

Java is distributed because it facilitates users to create distributed applications in Java. RMI and EJB are used for creating distributed applications. This feature of Java makes us able to access files by calling the methods from any machine on the internet.

### **Multi-threaded**

A thread is like a separate program, executing concurrently. We can write Java programs that deal with many tasks at once by defining multiple threads. The main advantage of multi-threading is that it doesn't occupy memory for each thread. It shares a common memory area. Threads are important for multi-media, Web applications, etc.

### **Dynamic**

Java is a dynamic language. It supports the dynamic loading of classes. It means classes are loaded on demand. It also supports functions from its native languages, i.e., C and C++. Java supports dynamic compilation and automatic memory management (garbage collection).

## What is Bytecode in Java?

### Overview

Bytecode in Java is a set of instructions for the Java Virtual Machine that is responsible for interpreting them. Upon compiling a Java program, Java bytecode is generated that can be executed on any platform (platform-independent) using the JVM. It is present in a .class file.

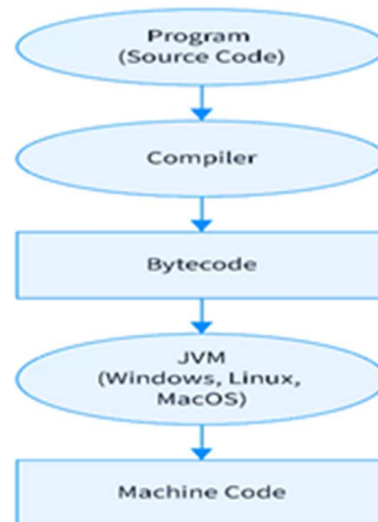
Can you imagine a world where we have to write code multiple times for each device with varying specifications like operating system, processor architecture, etc.? It will be a tedious process. Keeping these things in mind, Java developers came up with the concept of Write Once Read Anywhere, which got incorporated as a feature in the Java language. To achieve this, a code known as bytecode gets generated after compiling our Java source code. We will now discuss the same.

Bytecode in Java is a set of instructions for the Java Virtual Machine. Java Virtual Machine, abbreviated as JVM, enables a computer to run code written in Java. When a Java program is compiled, the bytecode gets generated. It is equivalent to the assembler in C++.

Bytecode is a platform-independent set of instructions primarily because it is interpreted and executed by the JVM. Code that runs on multiple computer architectures without any modification is called machine or platform-independent code.

Bytecode lies in-between low-level and high-level sets of instructions. Since we write our code in a high-level language, it gets compiled into bytecode, and later, JVM interprets it into machine code, a low-level set of instructions ready to be executed. Hence, we call bytecode a code between low-level and high-level language.

### How does it works?



- The code we write in Java is called the source code, which is written in a high-level language. A high-level language is a programmer-friendly language with statements written in English and is closer to human languages. The extension of the Java file is ".java".
- When we compile the program, the [compiler](#) compiles the ".java" file and generates a ".class" file. It contains the bytecode.
- The bytecode allows us to run the ".class" file on any other platform.
- But this bytecode requires an interpreter to execute it. Here the JVM comes into the picture. JVM has an interpreter. It executes the code piece by piece, i.e., one statement at a time, until it finds an error or is done with executing the end of the

Hence, bytecode is said to be between low-level language and high-level language.

Let us understand the above process with an explanation:

- The code we write in Java is called the source code, which is written in a high-level language. A high-level language is a programmer-friendly language with statements written in English and is closer to human languages. The extension of the Java file is ".java".
- When we compile the program, the [compiler](#) compiles the ".java" file and generates a ".class" file. It contains the bytecode.
- The bytecode allows us to run the ".class" file on any other platform.
- But this bytecode requires an interpreter to execute it. Here the JVM comes into the picture. JVM has an interpreter. It executes the code piece by piece, i.e., one statement at a time, until it finds an error or is done with executing the end of the code.

Hence, bytecode is said to be between low-level language and high-level language.

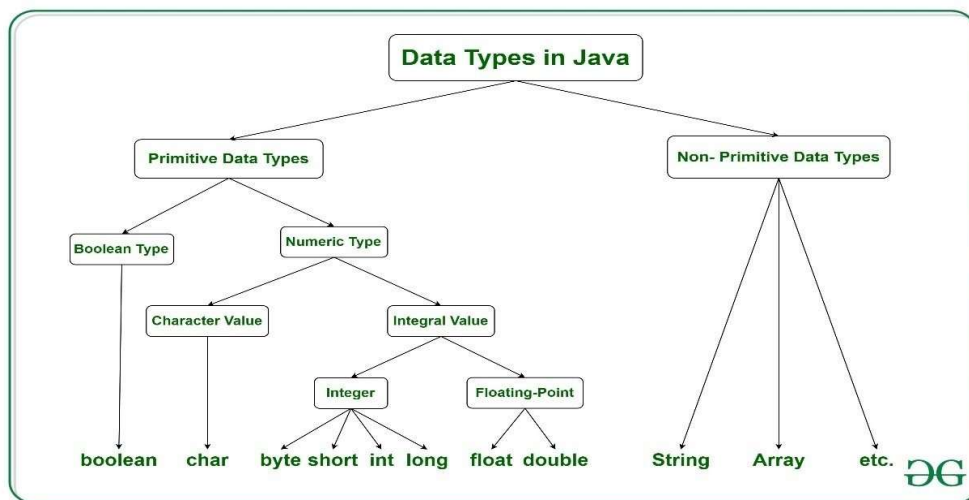
### **Java Data Types**

Java is statically typed and also a strongly typed language because, in Java, each type of data (such as integer, character, hexadecimal, packed decimal, and so forth) is predefined as part of the programming language and all constants or variables defined for a given program must be described with one of the data types.

#### **What are Data Types in Java?**

Data types in Java are of different sizes and values that can be stored in the variable that is made as per convenience and circumstances to cover up all test cases. Java has two categories in which data types are segregated

1. Primitive Data Type: such as boolean, char, int, short, byte, long, float, and double
2. Non-Primitive Data Type or Object Data type: such as String, Array, etc.



### Primitive Data Types in Java

Primitive data are only single values and have no special capabilities. There are 8 primitive data types. They are depicted below in tabular format below as follows:

| Type    | Description             | Default | Size    | Example Literals                       | Range of values                                       |
|---------|-------------------------|---------|---------|----------------------------------------|-------------------------------------------------------|
| boolean | true or false           | false   | 1 bit   | true, false                            | true, false                                           |
| byte    | twos-complement integer | 0       | 8 bits  | (none)                                 | -128 to 127                                           |
| char    | Unicode character       | \u0000  | 16 bits | 'a', '\u0041', '\101', '\\', '\n', '?' | characters representation of ASCII values<br>0 to 255 |
| short   | twos-complement integer | 0       | 16 bits | (none)                                 | -32,768 to 32,767                                     |

| Type   | Description             | Default | Size    | Example Literals                   | Range of values                                               |
|--------|-------------------------|---------|---------|------------------------------------|---------------------------------------------------------------|
| int    | twos-complement integer | 0       | 32 bits | -2,-1,0,1,2                        | -2,147,483,648<br>to<br>2,147,483,647                         |
| long   | twos-complement integer | 0       | 64 bits | -2L,-1L,0L,1L,2L                   | -9,223,372,036,854,775,808<br>to<br>9,223,372,036,854,775,807 |
| float  | IEEE 754 floating point | 0.0     | 32 bits | 1.23e100f, -1.23e-100f, .3f, 3.14F | upto 7 decimal digits                                         |
| double | IEEE 754 floating point | 0.0     | 64 bits | 1.23456e300d, -123456e-300d, 1e1d  | upto 16 decimal digits                                        |

Let us discuss and implement each one of the following data types that are as follows:

### 1. Boolean Data Type

Boolean data type represents only one bit of information either true or false which is intended to represent the two truth values of logic and Boolean algebra, but the size of the boolean data type is virtual machine-dependent. Values of type boolean are not converted implicitly or explicitly (with casts) to any other type. But the programmer can easily write conversion code.

Syntax:

```
boolean booleanVar;
```

Size: Virtual machine dependent

### 2. Byte Data Type

The byte data type is an 8-bit signed two's complement integer. The byte data type is useful for saving memory in large arrays.

Syntax:

```
byte byteVar;
```

Size: 1 byte (8 bits)

### 3. Short Data Type

The short data type is a 16-bit signed two's complement integer. Similar to byte, use a short to save memory in large arrays, in situations where the memory savings actually matters.

Syntax:

```
short shortVar;
```

Size: 2 bytes (16 bits)

### 4. Integer Data Type

It is a 32-bit signed two's complement integer.

Syntax:

```
int intVar;
```

Size: 4 bytes ( 32 bits )

Remember: In Java SE 8 and later, we can use the int data type to represent an unsigned 32-bit integer, which has a value in the range  $[0, 2^{32}-1]$ . Use the Integer class to use the int data type as an unsigned integer.

### 5. Long Data Type

The range of a long is quite large. The long data type is a 64-bit two's complement integer and is useful for those occasions where an int type is not large enough to hold the desired value. The size of the Long Datatype is 8 bytes (64 bits).

Syntax:

```
long longVar;
```

Remember: In Java SE 8 and later, you can use the long data type to represent an unsigned 64-bit long, which has a minimum value of 0 and a maximum value of  $2^{64}-1$ . The Long class also contains methods like comparing Unsigned, divide Unsigned, etc to support arithmetic operations for unsigned long.

### 6. Float Data Type

The float data type is a single-precision 32-bit IEEE 754 floating-point. Use a float (instead of double) if you need to save memory in large arrays of floating-point numbers. The size of the float data type is 4 bytes (32 bits).

Syntax:

```
float floatVar;
```

### 7. Double Data Type

The double data type is a double-precision 64-bit IEEE 754 floating-point. For decimal values, this data type is generally the default choice. The size of the double data type is 8 bytes or 64 bits.

Syntax:

```
double doubleVar;
```

Note: Both float and double data types were designed especially for scientific calculations, where approximation errors are acceptable. If accuracy is the most prior concern then, it is recommended not to use these data types and use BigDecimal class instead. It is recommended to go through [rounding off errors in java](#).

## 8. Char Data Type

The char data type is a single 16-bit Unicode character with the size of 2 bytes (16 bits).

Syntax:  
char charVar;

## Non-Primitive Data Type or Reference Data Types

The **Reference Data Types** will contain a memory address of variable values because the reference types won't store the variable value directly in memory. They are strings, objects, arrays, etc.

### 1. Strings

[Strings](#) are defined as an array of characters. The difference between a character array and a string in Java is, that the string is designed to hold a sequence of characters in a single variable whereas, a character array is a collection of separate char-type entities. Unlike C/C++, Java strings are not terminated with a null character.

**Syntax:** Declaring a string

<String\_Type> <string\_variable> = "<sequence\_of\_string>";

#### **Example:**

```
// Declare String without using new operator
```

```
String s = "GeeksforGeeks";
```

```
// Declare String using new operator
```

```
String s1 = new String("GeeksforGeeks");
```

### 2. Class

A [class](#) is a user-defined blueprint or prototype from which objects are created. It represents the set of properties or methods that are common to all objects of one type. In general, class declarations can include these components, in order:

1. **Modifiers:** A class can be public or has default access. Refer to [access specifiers for classes or interfaces in Java](#)
2. **Class name:** The name should begin with an initial letter (capitalized by convention).
3. **Superclass(if any):** The name of the class's parent (superclass), if any, preceded by the keyword extends. A class can only extend (subclass) one parent.
4. **Interfaces(if any):** A comma-separated list of interfaces implemented by the class, if any, preceded by the keyword implements. A class can implement more than one interface.
5. **Body:** The class body is surrounded by braces, { }.

### 3. Object

An [Object](#) is a basic unit of Object-Oriented Programming and represents real-life entities. A typical Java program creates many objects, which as you know, interact by invoking methods. An object consists of :

1. **State:** It is represented by the attributes of an object. It also reflects the properties of an object.
2. **Behavior:** It is represented by the methods of an object. It also reflects the response of an object to other objects.
3. **Identity:** It gives a unique name to an object and enables one object to interact with other objects.

### 4. Interface

Like a class, an [interface](#) can have methods and variables, but the methods declared in an interface are by default abstract (only method signature, no body).

- Interfaces specify what a class must do and not how. It is the blueprint of the class.
- An Interface is about capabilities like a Player may be an interface and any class implementing Player must be able to (or must implement) move(). So it specifies a set of methods that the class has to implement.
- If a class implements an interface and does not provide method bodies for all functions specified in the interface, then the class must be declared abstract.
- A Java library example is [Comparator Interface](#). If a class implements this interface, then it can be used to sort a collection.

### 5. Array

An [Array](#) is a group of like-typed variables that are referred to by a common name. Arrays in Java work differently than they do in C/C++. The following are some important points about Java arrays.

- In Java, all arrays are dynamically allocated. (discussed below)
- Since arrays are objects in Java, we can find their length using member length. This is different from C/C++ where we find length using size.
- A Java array variable can also be declared like other variables with [] after the data type.
- The variables in the array are ordered and each has an index beginning with 0.
- Java array can also be used as a static field, a local variable, or a method parameter.
- The **size** of an array must be specified by an int value and not long or short.
- The direct superclass of an array type is Object.
- Every array type implements the interfaces [Cloneable](#) and [java.io.Serializable](#).

## Variables

There are different types of variables in Java. They are as follows:

### **1. Instance Variables (Non-Static Fields)**

Objects store their individual states in “non-static fields”, that is, fields declared without the static keyword.

Non-static fields are also known as instance variables because their values are unique to each instance of a class. For example, the currentSpeed of one bicycle is independent from the currentSpeed of another.

## 2. Class Variables (Static Fields)

A class variable is any field declared with the static modifier; this tells the compiler that there is exactly one copy of this variable in existence, regardless of how many times the class has been instantiated. A field defining the number of gears for a particular kind of bicycle could be marked as static since, conceptually, the same number of gears will apply to all instances. The code `static int numGears = 6;` would create such a static field.

## 3. Local Variables

A method stores its temporary state in local variables. The syntax for declaring a local variable is similar to declaring a field (for example, `int count = 0;`). There is no special keyword designating a variable as local; that determination comes entirely from the location in which the variable is declared—between the opening and closing braces of a method. As such, local variables are only visible to the methods in which they are declared; they are not accessible from the rest of the class.

## 4. Parameters

They are the variables that are passed to the methods of a class.

### Variable Declaration :

Identifiers are the names of variables. They must be composed of only letters, numbers, the underscore, and the dollar sign (\$). They cannot contain white spaces. Identifiers may only begin with a letter, the underscore, or the dollar sign. A variable cannot begin with a number. All variable names are case sensitive.

### **Syntax for variable declaration**

`datatype1 variable1, datatype2 variable2, ... datatypeN variablen;`

For example:

`int a, char ch;`

### **Variable Initialisation :**

Variables can be assigned values in the following way: `Variablename = value;`

For example; ch='a';  
a=0;

## **Java Arrays**

An array is a group of variables that share the same data type, and are referred to by a common name. Arrays of any type can be created and may have one or more dimensions.

A specific element in an array is accessed by its index. The array index ranges from 0 to n-1; therefore, in an array of size 10, the first element is stored at index 0 and the last or the 10th element at index 9.

The following program, `Printarr`, creates an array of integers, puts some values in it, and prints each value to standard output.

```
class Printarr {
public static void main(String[] args) {
// declares an array of integers int[ ] A;

// allocates memory for 5 integers A = new
int[5];

// initialize elements A[0] = 15; //first
element

    A[1] = 20; //second element
    A[2] = 25; //third element
    A[3] = 30; //fourth element
    A[4] = 50; //fifth element

    System.out.println("Element at index 0: "
        + A[0]); System.out.println("Element at
        index 1: "
        + A[1]);
    System.out.println("Element at index 2: "
        + A[2]);
    System.out.println("Element at index 3: "
        + A[3]);
    System.out.println("Element at index 4: "
        + A[4]);
    }
}
```

The output from this program is:

```
at index 0: 15
at index 1: 20
at index 2: 25
at index 3: 30
at index 4: 50
```

Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value.

To declare an array, define the variable type with **square brackets**:

```
String[] cars;
```

We have now declared a variable that holds an array of strings. To insert values to it, you can place the values in a comma-separated list, inside curly braces:

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
```

To create an array of integers, you could write:

```
int[] myNum = {10, 20, 30, 40};
```

### Access the Elements of an Array

You can access an array element by referring to the index number.

This statement accesses the value of the first element in cars:

#### **Example**

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
System.out.println(cars[0]);
// Outputs Volvo
```

**Note:** Array indexes start with 0: [0] is the first element. [1] is the second element, etc.

### Change an Array Element

To change the value of a specific element, refer to the index number:

#### **Example**

```
cars[0] = "Opel";
```

#### **Example**

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
cars[0] = "Opel";
```

```
System.out.println(cars[0]);  
// Now outputs Opel instead of Volvo
```

## Array Length

To find out how many elements an array has, use the **length** property:

### **Example**

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};  
System.out.println(cars.length);  
// Outputs 4
```

## **Loop Through an Array**

You can loop through the array elements with the **for** loop, and use the **length** property to specify how many times the loop should run.

The following example outputs all elements in the **cars** array:

### **Example**

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};  
for (int i = 0; i < cars.length; i++) {  
    System.out.println(cars[i]);  
}
```

```
o/p  
Volvo  
BMW  
Ford  
Mazda
```

## Multidimensional Arrays

A multidimensional array is an array of arrays.

Multidimensional arrays are useful when you want to store data as a tabular form, like a table with rows and columns.

To create a two-dimensional array, add each array within its own set of **curly braces**:

### **Example:-**

```
int[][] myNumbers = { {1, 2, 3, 4}, {5, 6, 7} };
```

**myNumbers** is now an array with two arrays as its elements.

## Access Elements

To access the elements of the **myNumbers** array, specify two indexes: one for the array, and one for the element inside that array. This example accesses the third element (2) in the second array (1) of myNumbers:

### Example

```
int[][] myNumbers = { {1, 2, 3, 4}, {5, 6, 7} };
```

```
System.out.println(myNumbers[1][2]); // Outputs 7
```

**Remember that:** Array indexes start with 0: [0] is the first element. [1] is the second element, etc.

## Change Element Values

You can also change the value of an element:

### Example

```
int[][] myNumbers = { {1, 2, 3, 4}, {5, 6, 7} };
```

```
myNumbers[1][2] = 9;
```

```
System.out.println(myNumbers[1][2]); // Outputs 9 instead of 7
```

## Operators in Java

**Operator** in [Java](#) is a symbol that is used to perform operations. For example: +, -, \*, / etc.

There are many types of operators in Java which are given below:

- Unary Operator,
- Arithmetic Operator,
- Shift Operator,
- Relational Operator,
- Bitwise Operator,
- Logical Operator,
- Ternary Operator and
- Assignment Operator.

# Java Operator Precedence

| Operator Type | Category             | Precedence                                                    |
|---------------|----------------------|---------------------------------------------------------------|
| Unary         | postfix              | <i>expr</i> ++ <i>expr</i> --                                 |
|               | prefix               | ++ <i>expr</i> -- <i>expr</i> + <i>expr</i> - <i>expr</i> ~ ! |
| Arithmetic    | multiplicative       | * / %                                                         |
|               | additive             | -                                                             |
| Shift         | shift                | << >> >>>                                                     |
| Relational    | comparison           | > <= >= instanceof                                            |
|               | equality             | == !=                                                         |
| Bitwise       | bitwise AND          | &                                                             |
|               | bitwise exclusive OR |                                                               |
|               | bitwise inclusive OR |                                                               |
| Logical       | logical AND          | &&                                                            |
|               | logical OR           |                                                               |
| Ternary       | ternary              | :                                                             |
| Assignment    | assignment           | + = - = * = / = % = & = ^ =   = << = >> = >>> =               |

## Classes and Objects in Java

In Java, classes and objects are basic concepts of Object-Oriented Programming (OOPs) that are used to represent real-world concepts and entities. The class represents a group of objects having similar properties and behavior. For example, the animal type **Dog** is a class while a particular dog named **Tommy** is an object of the **Dog** class.

In this article, we will discuss Java objects and classes and how to implement them in our program.

### Java Classes

A class in Java is a set of objects which shares common characteristics/ behavior and common properties/ attributes. It is a user-defined blueprint or prototype from which objects are created. For example, Student is a class while a particular student named Ravi is an object.

### Properties of Java Classes

1. Class is not a real-world entity. It is just a template or blueprint or prototype from which objects are created.
2. Class does not occupy memory.
3. Class is a group of variables of different data types and a group of methods.
4. A Class in Java can contain:
  - Data member
  - Method
  - Constructor
  - Nested Class
  - Interface

### Class Declaration in Java

```
access_modifier class <class_name>
{
    data member;
    method;
    constructor;
    nested class;
    interface;
}
```

### Java Objects

An object in Java is a basic unit of Object-Oriented Programming and represents real-life entities. Objects are the instances of a class that are created to use the attributes and methods of a class. A typical Java program creates many objects, which as you know, interact by invoking methods. An object consists of :

1. **State:** It is represented by attributes of an object. It also reflects the properties of an object.
2. **Behavior:** It is represented by the methods of an object. It also reflects the response of an object with other objects.
3. **Identity:** It gives a unique name to an object and enables one object to interact with other objects.

Example of an object: dog



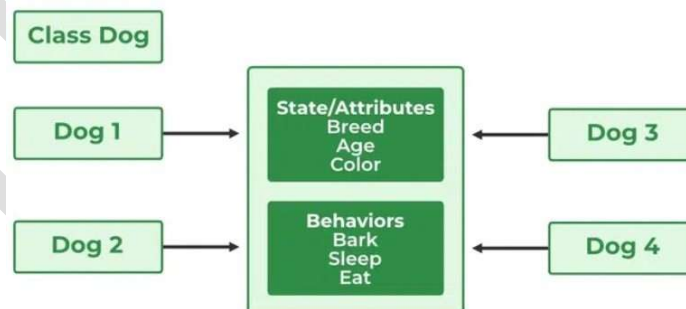
*Java Objects*

Objects correspond to things found in the real world. For example, a graphics program may have objects such as “circle”, “square”, and “menu”. An online shopping system might have objects such as “shopping cart”, “customer”, and “product”.

Declaring Objects (Also called instantiating a class)

When an object of a class is created, the class is said to be **instantiated**. All the instances share the attributes and the behavior of the class. But the values of those attributes, i.e. the state are unique for each object. A single class may have any number of instances.

**Example:**



*Java Object Declaration*

As we declare variables like (type name;). This notifies the compiler that we will use the name to refer to data whose type is type. With a primitive variable, this declaration also reserves the proper amount of memory for the variable. So for reference variables, the type must be strictly a concrete class name. In general, we **can't** create objects of an abstract class or an interface.

Dog tuffy;

If we declare a reference variable(tuffy) like this, its value will be undetermined(null) until an object is actually created and assigned to it. Simply declaring a reference variable does not create an object.

### Java Methods

The method in java or Methods of Java is a collection of statements that perform some specific task and return the result to the caller. A Java method can perform some specific task without returning anything. Java Methods allow us to **reuse** the code without retyping the code. In Java, every method must be part of some class that is different from languages like C, C++, and Python.

1. A method is like a function i.e. used to expose the behavior of an object.
2. It is a set of codes that perform a particular task.

Syntax of Method

```
<access_modifier> <return_type> <method_name> ( list_of_parameters )
{
    //body
}
```

### **Advantage of Method**

- Code Reusability
- Code Optimization

***Note:** Methods are time savers and help us to reuse the code without retyping the code.*

### **Method Declaration**

In general, method declarations have 6 components:

**1. Modifier:** It defines the **access type** of the method i.e. from where it can be accessed in your application. In Java, there 4 types of access specifiers.

- **public:** It is accessible in all classes in your application.
- **protected:** It is accessible within the class in which it is defined and in its subclass/es
- **private:** It is accessible only within the class in which it is defined.
- **default:** It is declared/defined without using any modifier. It is accessible within the same class and package within which its class is defined.

***Note:** It is **Optional** in syntax.*

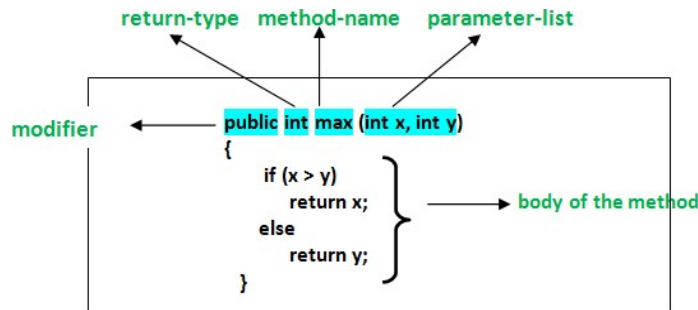
**2. The return type:** The data type of the value returned by the method or void if does not return a value. It is **Mandatory** in syntax.

**3. Method Name:** the rules for field names apply to method names as well, but the convention is a little different. It is **Mandatory** in syntax.

**4. Parameter list:** Comma-separated list of the input parameters is defined, preceded by their data type, within the enclosed parenthesis. If there are no parameters, you must use empty parentheses (). It is **Optional** in syntax.

**5. Exception list:** The exceptions you expect by the method can throw, you can specify these exception(s). It is **Optional** in syntax.

**6. Method body:** it is enclosed between braces. The code you need to be executed to perform your intended operations. It is **Optional** in syntax.



### Types of Methods in Java

*There are two types of methods in Java:*

#### 1. Predefined Method

In Java, predefined methods are the method that is already defined in the Java class libraries is known as predefined methods. It is also known as the standard library method or built-in method. We can directly use these methods just by calling them in the program at any point.

#### 2. User-defined Method

The method written by the user or programmer is known as a user-defined method. These methods are modified according to the requirement.

### *Two Ways to Create Method in Java*

There are two ways to create a method in Java:

**1. Instance Method:** Access the instance data using the object name. Declared inside a class.

*Syntax:*

```
// Instance Method
void method_name()
{ body // instance area
}
```

**2. Static Method:** Access the static data using class name. Declared inside class with **static** keyword.

*Syntax:*

```
//Static Method
static void method_name()
{ body // static area }
```

### Method Signature

It consists of the method name and a parameter list (number of parameters, type of the parameters, and order of the parameters). The return type and exceptions are not considered as part of it.

Method Signature of the above function:

max(int x, int y) Number of parameters is 2, Type of parameter is int.

### How to Name a Method?

A method name is typically a single word that should be a **verb** in lowercase or a multi-word, that begins with a **verb** in lowercase followed by an **adjective, noun.....** After the first word, the first letter of each word should be capitalized.

#### **Rules to Name a Method**

- While defining a method, remember that the method name must be a **verb** and start with a **lowercase** letter.
- If the method name has more than two words, the first name must be a verb followed by an adjective or noun.
- In the multi-word method name, the first letter of each word must be in **uppercase** except the first word. For example, findSum, computeMax, setX, and getX.

Generally, a method has a unique name within the class in which it is defined but sometimes a method might have the same name as other method names within the same class as [method overloading is allowed in Java](#).

### Method Calling

The method needs to be called for use its functionality. There can be three situations when a method is called:

A method returns to the code that invoked it when:

- It completes all the statements in the method
- It reaches a return statement
- Throws an exception

#### **Example:**

```
// Java Program to Illustrate Methods
// Importing required classes
import java.io.*;
// Class 1
// Helper class
class Addition {
// Initially taking sum as 0
```

```

// as we have not started computation
int sum = 0;
// Method
// To add two numbers
public int addTwoInt(int a, int b)
{ // Adding two
  integer value
  sum = a + b;
  // Returning summation of two values
  return sum;
}
}
// Class 2
// Helper class
class GFG {
  // Main driver method
  public static void main(String[] args) {
    // Creating object of class 1 inside main() method
    Addition add = new Addition();
    // Calling method of above class
    // to add two integer
    // using instance created
    int s = add.addTwoInt(1, 2);
    // Printing the sum of two numbers
    System.out.println("Sum of two integer values :+ s);
  }
}

```

### Output

Sum of two integer values :3

## Java Constructors

Java constructors or constructors in Java is a terminology used to construct something in our programs. A constructor in Java is a **special method** that is used to initialize objects. The constructor is called when an object of a class is created. It can be used to set initial values for object attributes.

### What are Constructors in Java?

In Java, Constructor is a block of codes similar to the method. It is called when an instance of the class is created. At the time of calling the constructor, memory for the object is allocated in the memory. It is a special type of method that is used to initialize the object. Every time an object is created using the new() keyword, at least one constructor is called. Example

```

// Java Program to demonstrate
// Constructor
import java.io.*;
class Geeks {
  // Constructor
  Geeks()
  {
    super();
    System.out.println("Constructor Called");
  }
}
// main function

```

```

public static void main(String[] args)
{
    Geeks geek = new Geeks();
}

```

## Output

Constructor Called

**Note:** It is not necessary to write a constructor for a class. It is because the java compiler creates a default constructor (constructor with no arguments) if your class doesn't have any.

## How Java Constructors are Different From Java Methods?

- Constructors must have the same name as the class within which it is defined it is not necessary for the method in Java.
- Constructors do not return any type while method(s) have the return type or **void** if does not return any value.
- Constructors are called only once at the time of Object creation while method(s) can be called any number of times.

Now let us come up with the syntax for the constructor being invoked at the time of object or instance creation.

```

class Geek
{
.....
// A Constructor
Geek() {
}
.....
}
// We can create an object of the above class
// using the below statement. This statement
// calls above constructor.
Geek obj = new Geek();

```

The first line of a constructor is a call to `super()` or `this()`, (a call to a constructor of a super-class or an overloaded constructor), if you don't type in the call to `super` in your constructor the compiler will provide you with a non-argument call to `super` at the first line of your code, the `super` constructor must be called to create an object:

If you think your class is not a subclass it actually is, every class in Java is the subclass of a class **object** even if you don't say `extends object` in your class definition.

## Need of Constructors in Java

Think of a Box. If we talk about a box class then it will have some class variables (say length, breadth, and height). But when it comes to creating its object(i.e Box will now exist in the computer's memory), then can a box be there with no value defined for its dimensions? The answer is **No**.

So constructors are used to assign values to the class variables at the time of object creation, either explicitly done by the programmer or by Java itself (default constructor).

### When Constructor is called?

Each time an object is created using a **new()** keyword, at least one constructor (it could be the default constructor) is invoked to assign initial values to the **data members** of the same class. Rules for writing constructors are as follows:

- The constructor(s) of a class must have the same name as the class name in which it resides.
- A constructor in Java can not be abstract, final, static, or Synchronized.
- Access modifiers can be used in constructor declaration to control its access i.e which other class can call the constructor.

So by far, we have learned constructors are used to initialize the object's state. Like [methods](#), a constructor also contains a collection of statements(i.e. instructions) that are executed at the time of Object creation.

### Types of Constructors in Java

Now is the correct time to discuss the types of the constructor, so primarily there are three types of constructors in Java are mentioned below:

- **Default Constructor**
- **Parameterized Constructor**
- **Copy Constructor**

#### 1. Default Constructor in Java

A constructor that has no parameters is known as default the constructor. A default constructor is invisible. And if we write a constructor with no arguments, the compiler does not create a default constructor. It is taken out. It is being overloaded and called a parameterized constructor. The default constructor changed into the parameterized constructor. But Parameterized constructor can't change the default constructor.

#### Example:

```
// Java Program to demonstrate
// Default Constructor
import java.io.*;

// Driver class
class GFG {

    // Default Constructor
    GFG() { System.out.println("Default constructor"); }

    // Driver function
    public static void main(String[] args)
    {
        GFG hello = new GFG();
    }
}
```

#### Output

Default constructor

***Note:** Default constructor provides the default values to the object like 0, null, etc. depending on the type.*

## **2. Parameterized Constructor in Java**

A constructor that has parameters is known as parameterized constructor. If we want to initialize fields of the class with our own values, then use a parameterized constructor.

### **Example:**

```
// Java Program for Parameterized Constructor
import java.io.*;
class Geek {
    // data members of the class.
    String name;
    int id;
    Geek(String name, int id)
    {
        this.name = name;
        this.id = id;
    }
}
class GFG {
    public static void main(String[] args)
    {
        // This would invoke the parameterized constructor.
        Geek geek1 = new Geek("avinash", 68);
        System.out.println("GeekName :"+ geek1.name
            + " and GeekId :"+ geek1.id);
    }
}
```

### **Output**

GeekName :avinash and GeekId :68

## **3. Copy Constructor in Java**

Unlike other constructors copy constructor is passed with another object which copies the data available from the passed object to the newly created object.

### **Example:**

```
// Java Program for Copy Constructor
import java.io.*;
class Geek {
    // data members of the class.
    String name;
    int id;
    // Parameterized Constructor
    Geek(String name, int id)
    {
        this.name = name;
        this.id = id;
    }
}
```

```

    }
    // Copy Constructor
    Geek(Geek obj2)
    {
        this.name = obj2.name;
        this.id = obj2.id;
    }
}
class GFG {
    public static void main(String[] args)
    {
        // This would invoke the parameterized constructor.
        System.out.println("First Object");
        Geek geek1 = new Geek("avinash", 68);
        System.out.println("GeekName :" + geek1.name + " and GeekId :" + geek1.id);
        System.out.println();
        // This would invoke the copy constructor.
        Geek geek2 = new Geek(geek1);
        System.out.println("Copy Constructor used Second Object");
        System.out.println("GeekName :" + geek2.name + " and GeekId :" + geek2.id);
    }
}

```

### Output

First Object

GeekName :avinash and GeekId :68

Copy Constructor used Second Object

GeekName :avinash and GeekId :68

## Constructor overloading in Java

In Java, we can overload constructors like methods. The constructor overloading can be defined as the concept of having more than one constructor with different parameters so that every constructor can perform a different task.

Consider the following [Java](#) program, in which we have used different constructors in the class.

### Example

1. **public class** Student {
2. *//instance variables of the class*
3. **int** id;
4. String name;
5. Student(){
6. System.out.println("this a default constructor");

```

7. }
8. Student(int i, String n){
9.     id = i;
10.    name = n;
11. }
12. public static void main(String[] args) {
13.    //object creation
14.    Student s = new Student();
15.    System.out.println("\nDefault Constructor values: \n");
16.    System.out.println("Student Id : "+s.id + "\nStudent Name : "+s.name);
17.    System.out.println("\nParameterized Constructor values: \n");
18.    Student student = new Student(10, "David");
19.    System.out.println("Student Id : "+student.id + "\nStudent Name : "+student.name);
20. }
21. }

```

Output:

this a default constructor

Default Constructor values:

Student Id: 0 Student

Name: null

Parameterized Constructor values:

Student Id: 10 Student

Name: David

In the above example, the Student class constructor is overloaded with two different constructors, I.e., default and parameterized.

### Why Do We Need Constructor Overloading in Java?

Constructor overloading in Java serves several important purposes that contribute to code flexibility, maintainability, and ease of use. Here are the key reasons why we need constructor overloading:

- **Multiple Initialization Options:** Different scenarios may require different sets of initial values for objects. Constructor overloading allows you to provide multiple ways to initialize objects based on the needs of the user or the context in which the object is created. This leads to a more versatile and adaptable code.
- **Code Readability:** Constructor overloading enhances code readability by providing constructors with descriptive parameter names. This makes it easier for other developers (and your future self) to understand the purpose and usage of each constructor without referring to extensive documentation.

- **Default Values:** With constructor overloading, you can set default values for certain parameters. This is especially useful when some attributes of an object may not be known or relevant during object creation. Constructors with default values simplify object creation by reducing the number of required parameters.
- **Encapsulation and Abstraction:** By offering multiple constructors, you can do [encapsulation](#) to the details of object initialization within the class. This way, the users of your class don't need to know the internal complexities; they can create objects using the constructors you've provided, making your class easier to use and understand.

## Garbage Collection in Java

Garbage collection in Java is the process by which Java programs perform automatic memory management. Java programs compile to bytecode that can be run on a Java Virtual Machine, or JVM for short. When Java programs run on the JVM, objects are created on the heap, which is a portion of memory dedicated to the program. Eventually, some objects will no longer be needed. The garbage collector finds these unused objects and deletes them to free up memory.

### How can an object be unreferenced?

- By nulling the reference
  - By assigning a reference to another
  - By anonymous object etc.

#### 1) By nulling a reference:

1. Employee e=**new** Employee();
2. e=**null**;

#### 2) By assigning a reference to another:

1. Employee e1=**new** Employee();
2. Employee e2=**new** Employee();
3. e1=e2; **//now the first object referred by e1 is available for garbage collection**

#### 3) By anonymous object:

1. **new** Employee();

### finalize() method

The finalize() method is invoked each time before the object is garbage collected. This method can be used to perform cleanup processing. This method is defined in Object class as:

1. **protected void** finalize(){}

### gc() method

The gc() method is used to invoke the garbage collector to perform cleanup processing. The gc() is found in System and Runtime classes.

1. **public static void** gc(){}

### Simple Example of garbage collection in java

```
public class TestGarbage1 {  
    public void finalize(){System.out.println("object is garbage collected");} public  
        static void main(String args[]){  
            TestGarbage1 s1=new TestGarbage1();  
            TestGarbage1 s2=new TestGarbage1();  
            s1=null;  
            s2=null;  
            System.gc();  
        }  
}
```

**Output:-**

object is garbage collected object is garbage collected

## Static Variables and Methods

- There is only one copy of a static variable or method for the whole class. For example, the main method is static because there should only be 1 main method.
- Static methods can be public or private.
- The static keyword is placed right after the public/private modifier and right before the type of variables and methods in their declarations.
- A static method does not have **this** variable, since it is not called with an object.

```
class ClassName {  
    // static variable  
    public static type variableName;  
  
    // static method  
    public static returnType methodName(parameters) {  
        // implementation not shown  
    }  
}  
  
// To call a static method or variable, use the Class Name  
System.out.println(ClassName.staticVariable);  
ClassName.staticMethod();
```

Static methods only have access to other static variables and static methods (unless the static method creates an object through which to access instance variables and methods). Static methods cannot access or change the values of instance variables or use the “this” reference (since there is no calling object for them), and static methods cannot call non-static methods. However, non-static methods have access to all variables (instance or static) and methods (static or non-static) in the class.

Since there is only 1 copy of a static variable or method, static variables are often used to count how many objects are generated. In the following class Person, there is a static variable called personCounter that is incremented each time the Person constructor is called to initialize a new Person object. The static method printCounter() prints out its value.

```

public class Person
{
    // instance variables
    private String name;
    private String email;
    private String phoneNumber;
    // Static counter variable
    public static int personCounter = 0;
    // static method to print out counter
    public static void printPersonCounter() {
        System.out.println("Person counter: " + personCounter);
    }
    // constructor: construct a Person copying in the data into the instance variables
    public Person(String initName, String initEmail, String initPhone)
    {
        name = initName; email =
        initEmail; phoneNumber =
        initPhone;
        personCounter++;
    }
    // toString() method
    public String toString()
    {
        return name + ": " + email + " " + phoneNumber;
    }
    // main method for testing
    public static void main(String[] args)
    {
        // call the constructor to create a new person
        Person p1 = new Person("Sana", "sana@gmail.com", "123-456-7890");
        Person p2 = new Person("Jean", "jean@gmail.com", "404 899-9955");
        Person.printPersonCounter();
    }
}

```

Person counter: 2

| Result | Expected          | Actual            | Notes                     |
|--------|-------------------|-------------------|---------------------------|
| Pass   | Person counter: 2 | Person counter: 4 | Expected output from main |

You got 1 out of 1 correct. 100.00%

### Static variables and static methods are two important concepts in Java.

Whenever a variable is declared as static, this means there is only one copy of it for the entire class, rather than each instance having its own copy. A static method means it can be called without creating an instance of the class.

Static variables and methods in Java provide several advantages, including memory efficiency, global access, object independence, performance, and code organization

In this article, you will learn how static variables work in Java, as well as why and how to use static methods.

### The Static Keyword in Java

The static keyword is one of the most essential features in the Java programming language.

We use it to define class-level variables and methods.

Here is an example of how to use the static keyword:

```
public class StaticKeyword
```

Example

```
{
    private static int count = 0; // static variable
    public static void printCount()
    { // static method
        System.out.println("Number of Example objects created so far: " + count);
    }
}
```

As you can see above, we declared the **count** variable as a static variable, while we declared the **printCount** method as a static method.

When a variable is declared static in Java programming, it means that the variable belongs to the class itself rather than to any specific instance of the class. This means that there is only one copy of the variable in memory, regardless of how many instances of the class are created.

Here's an example. Say we have a **Department** class that has a static variable called **numberOfWorker**. We declare and increment the static variable at the constructor level to show the value of the static variable whenever the class object is created.

```
public class Department
{
    public static int numberOfWorker= 0;
    public String name;
    public Department(String name)
    {
        this.name = name;
        numberOfWorker++; // increment the static variable every time a new
//Person is created
    }
}
```

The results of the above code show that as we create new Department objects, the static variable **numberOfWorker** retains its value.

When we print out the value of **numberOfWorker** in the console, we can see that it retains its value across all instances of the Department class. This is because there is only one copy of the variable in memory, and any changes to the variable will be reflected across all instances of the class.

```
Department dpt1 = new Department("Admin");
System.out.println(Department.numberOfWorker); // output: 1
```

```
Department dpt2 = new Department ("Finance");
System.out.println(Department.numberOfWorker); // output: 2
Department dpt3 = new Department ("Software");
```

```
System.out.println(Department.numberOfWorker); // output: 3
```

**We can also use the static keyword to define static methods.**

Static methods are methods that belong to the class rather than to any specific instance of the class. Static methods can be called directly on the class itself without needing to create an instance of the class first. See the code below:

```
public class Calculation{
    public static int add(int a, int b)
    { return a + b;
    }
    public static int multiply(int a, int b)
    { return a * b;
    }
}
```

In the above code, the Calculation class has two static methods. The declared static methods can be called directly on the Calculation class without creating an instance of the class first. That is to say, you do not need to create an object of the Calculation class before you access the static add and multiply classes.

```
int result = Calculation.add(5, 10);
System.out.println(result); // Output: 15
```

```
int result2 = Calculation.multiply(5, 10);
System.out.println(result2); // Output: 50
```

The main() method in Java is an example of a static method. The main() method is a special static method that is the entry point for Java applications. The Math class in Java also provides many static methods that perform mathematical operations.

```
public class Main
{
    public static void main(String[] args)
    {
        System.out.println("Hello, World!");
    }
}

int result = Math.max(5, 10);
System.out.println(result); // Output: 10
```

The above code shows that the entry point for Java applications is a static method. It also shows that the max() method is a static method of the Math class and does not require an instance of the Math class to be created.

As you can see, static methods can be useful in providing utility functions that do not necessitate the creation of a class object.

## **Final variable in Java**

A final variable can be explicitly initialized only once. A reference variable declared final can never be reassigned to refer to a different object.

However, the data within the object can be changed. So, the state of the object can be changed but not the reference.

With variables, the *final modifier* often is used with static to make the constant a class variable.

### **Example**

```
public class Test {  
    final int value = 10;  
    // The following are examples of declaring constants:  
    public static final int BOXWIDTH = 6;  
    static final String TITLE = "Manager";  
    public void changeValue() {  
        value = 12; // will give an error  
    }  
}
```

**OR**

### **Java final variable**

If you make any variable as final, you cannot change the value of final variable(It will be constant).

### **Example of final variable**

There is a final variable speedlimit, we are going to change the value of this variable, but It can't be changed because final variable once assigned a value can never be changed.

```
class Bike9  
{  
    final int speedlimit=90;//final variable  
    void run(){  
        speedlimit=400;  
    }  
    public static void main(String args[])  
    { Bike9 obj=new Bike9();  
      obj.run();  
    }  
} //end of class
```

**Compile by: javac Bike9.java**

## Java Command Line Arguments

- The java command-line argument is an argument i.e. passed at the time of running the java program.
- The arguments passed from the console can be received in the java program and it can be used as an input.
- So, it provides a convenient way to check the behavior of the program for the different values.
- You can pass N (1,2,3 and so on) numbers of arguments from the command prompt.

### Simple example of command-line argument in java

In this example, we are receiving only one argument and printing it. To run this java program, you must pass at least one argument from the command prompt.

```
class CommandLineExample
{
    public static void main(String args[]){
        System.out.println("Your first argument is: "+args[0]);
    }
}
```

compile by > javac CommandLineExample.java

run by > java CommandLineExample sonoo

**Output: Your first argument is: sonoo**

## Super Keyword in Java

The **super** keyword in Java is a reference variable which is used to refer immediate parent class object. Whenever you create the instance of subclass, an instance of parent class is created implicitly which is referred by super reference variable.

### Usage of Java super Keyword

1. super can be used to refer immediate parent class instance variable.
2. super can be used to invoke immediate parent class method.
3. super() can be used to invoke immediate parent class constructor.

#### **1) super is used to refer immediate parent class instance variable.**

We can use super keyword to access the data member or field of parent class. It is used if parent class and child class have same fields.

1. **class** Animal{
2. String color="white";
3. }
4. **class** Dog **extends** Animal{

```

5. String color="black";
6. void printColor(){
7. System.out.println(color);//prints color of Dog class
8. System.out.println(super.color);//prints color of Animal class
9. }
10. }
11. class TestSuper1 {
12. public static void main(String args[]){
13. Dog d=new Dog();
14. d.printColor();
15. }}

```

**Output:**

```
black white
```

In the above example, Animal and Dog both classes have a common property color. If we print color property, it will print the color of current class by default. To access the parent property, we need to use super keyword.

## 2) super can be used to invoke parent class method

The super keyword can also be used to invoke parent class method. It should be used if subclass contains the same method as parent class. In other words, it is used if method is overridden.

```

class Animal
{
    void eat(){System.out.println("eating...");}
}

class Dog extends Animal
{
    void eat()
    {
        System.out.println("eating bread...");
    }
    void bark()
    {
        System.out.println("barking...");
    }
    void work()

```

```
{  
    super.eat();  
    bark()  
}
```

```
class TestSuper2 {  
    public static void main(String args[])  
    {  
        Dog d = new Dog();  
        d.work();  
    }  
}
```

**Output:**

```
eating... barking...
```

In the above example Animal and Dog both classes have eat() method if we call eat() method from Dog class, it will call the eat() method of Dog class by default because priority is given to local.

To call the parent class method, we need to use super keyword.

**3) super is used to invoke parent class constructor.**

The super keyword can also be used to invoke the parent class constructor. Let's see a simple example:

```
1. class Animal{
2.     Animal(){System.out.println("animal is created");}
3. }
4. class Dog extends Animal{
5.     Dog(){
6.         super();
7.         System.out.println("dog is created");
8.     }
9. }
10. class TestSuper3{
11.     public static void main(String args[]){
12.         Dog d=new Dog();
13.     }}
```

**Output:**

```
animal is created dog is created
```

## Final Keyword In Java

The **final** keyword in java is used to restrict the user. The java final keyword can be used in many context. Final can be:

1. variable
2. method
3. class

The final keyword can be applied with the variables, a final variable that have no value it is called blank final variable or uninitialized final variable. It can be initialized in the constructor only. The blank final variable can be static also which will be initialized in the static block only. We will have detailed learning of these. Let's first learn the basics of final keyword.

### 1) Java final variable

If you make any variable as final, you cannot change the value of final variable(It will be constant).

#### Example of final variable

There is a final variable speedlimit, we are going to change the value of this variable, but It can't be changed because final variable once assigned a value can never be changed.

1. **class** Bike9{
2. **final int** speedlimit=90;//final variable
3. **void** run(){
4. speedlimit=400;
5. }
6. **public static void** main(String args[]){
7. Bike9 obj=new Bike9();
8. obj.run();
9. }
10. }//end of class

**Output:Compile Time Error**

### 2) Java final method

If you make any method as final, you cannot override it.

#### Example of final method

```
class Bike
{
final void run()
{
System.out.println("running");
}
```

```
}  
class Honda extends Bike  
{  
void run()  
{  
System.out.println("running safely with 100kmph");  
}  
public static void main(String args[])  
{  
Honda honda= new Honda(); honda.run();  
}}  
}
```

Output: Compile Time Error

### 3) Java final class

If you make any class as final, you cannot extend it.

#### Example of final class

```
final class Bike{}  
class Honda1 extends Bike  
{  
    void run()  
    {  
        System.out.println("running safely with 100kmph");  
    }  
    public static void main(String args[])  
    {  
        Honda1 honda= new Honda1(); honda.run();  
    }  
}
```

**Output:Compile Time Error**

#### Q) Is final method inherited?

Ans) Yes, final method is inherited but you cannot override it. For Example:

```
class Bike  
{  
    final void run(){System.out.println("running...");}  
}  
class Honda2 extends Bike  
{  
    public static void main(String args[])  
    {  
        new Honda2().run();  
    }  
}  
  
Output: running...
```

## Java Wrapper Classes

Wrapper classes provide a way to use primitive data types (**int**, **boolean**, etc..) as objects. The table below shows the primitive type and the equivalent wrapper class:

| Primitive Data Type | Wrapper Class |
|---------------------|---------------|
| byte                | Byte          |
| short               | Short         |
| int                 | Integer       |
| long                | Long          |
| float               | Float         |
| double              | Double        |
| boolean             | Boolean       |
| char                | Character     |

Sometimes you must use wrapper classes, for example when working with Collection objects, such as **ArrayList**, where primitive types cannot be used (the list can only store objects):

### **Example**

```
ArrayList<int> myNumbers = new ArrayList<int>(); // Invalid
```

```
ArrayList<Integer> myNumbers = new ArrayList<Integer>(); // Valid
```

### Creating Wrapper Objects

To create a wrapper object, use the wrapper class instead of the primitive type. To get the value, you can just print the object:

#### **Example**

```
public class Main {  
    public static void main(String[] args)  
    { Integer myInt = 5; Double  
      myDouble = 5.99; Character  
      myChar = 'A';  
      System.out.println(myInt);  
      System.out.println(myDouble);  
      System.out.println(myChar);  
    }  
} o/p
```

```
5  
5.99  
A
```

### Exception in Java

Exception Handling is a mechanism to handle runtime errors such as `ClassNotFoundException`, `IOException`, `SQLException`, `RemoteException`, etc.

#### **Advantage of Exception Handling**

The core advantage of exception handling is **to maintain the normal flow of the application**. An exception normally disrupts the normal flow of the application; that is why we need to handle exceptions. Let's consider a scenario:

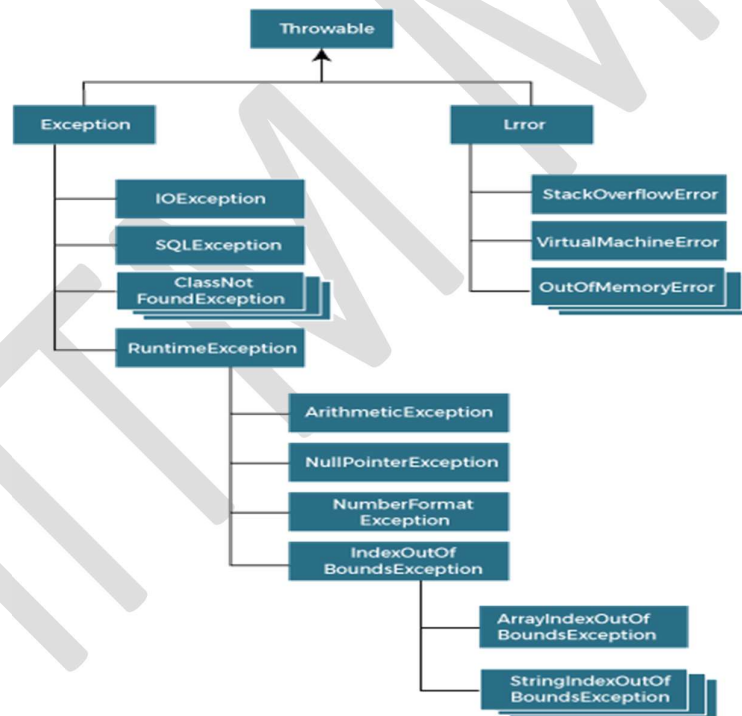
1. statement 1;
2. statement 2;
3. statement 3;
4. statement 4;

5. statement 5; *//exception occurs*
6. statement 6;
7. statement 7;
8. statement 8;
9. statement 9;
10. statement 10;

Suppose there are 10 statements in a Java program and an exception occurs at statement 5; the rest of the code will not be executed, i.e., statements 6 to 10 will not be executed. However, when we perform exception handling, the rest of the statements will be executed. That is why we use exception handling in [Java](#).

### Hierarchy of Java Exception classes

The java.lang.Throwable class is the root class of Java Exception hierarchy inherited by two subclasses: Exception and Error. The hierarchy of Java Exception classes is given below:



### Java Exception Keywords

Java provides five keywords that are used to handle the exception. The following table describes each.

| Keyword | Description                                                                                                                                                                               |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| try     | The "try" keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The try block must be followed by either catch or finally.   |
| catch   | The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.                |
| finally | The "finally" block is used to execute the necessary code of the program. It is executed whether an exception is handled or not.                                                          |
| throw   | The "throw" keyword is used to throw an exception.                                                                                                                                        |
| throws  | The "throws" keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It doesn't throw an exception. It is always used with method signature. |

### Java Exception Handling Example

Let's see an example of Java Exception Handling in which we are using a try-catch statement to handle the exception.

#### JavaExceptionExample.java

```

public class JavaExceptionExample{
    public static void main(String args[]){
        try{
            //code that may raise exception
            int data=100/0;
        }catch(ArithmeticException e){System.out.println(e);}
        //rest code of the program
        System.out.println("rest of the code...");
    }
}

```

## Output:

```
Exception in thread main java.lang.ArithmeticException:/ by zero
rest of the code...
```

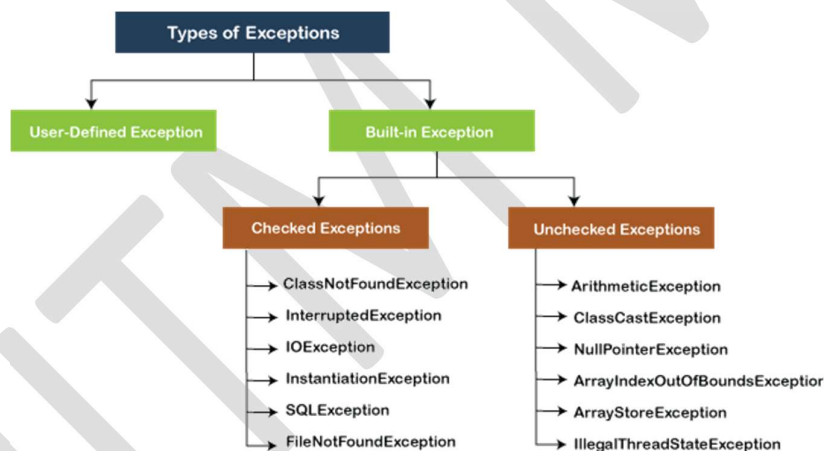
In the above example, 100/0 raises an `ArithmeticException` which is handled by a try-catch block.

## Types of Exception in Java

In Java, **exception** is an event that occurs during the execution of a program and disrupts the normal flow of the program's instructions. Bugs or errors that we don't want and restrict our program's normal execution of code are referred to as **exceptions**. In this section, we will focus on the **types of exceptions in Java** and the differences between the two.

Exceptions can be categorized into two ways:

1. Built-in Exceptions
  - Checked Exception
  - Unchecked Exception
2. User-Defined Exceptions



### Built-in Exception

**Exceptions** that are already available in **Java libraries** are referred to as **built-in exception**. These exceptions are able to define the error situation so that we can understand the reason of getting this error. It can be categorized into two broad categories, i.e., **checked exceptions** and **unchecked exception**.

### Checked Exception

**Checked** exceptions are called **compile-time** exceptions because these exceptions are checked at compile-time by the compiler. The compiler ensures whether the programmer handles the exception or not. The programmer should have to handle the exception; otherwise, the system has shown a compilation error.

### Unchecked Exceptions

The **unchecked** exceptions are just opposite to the **checked** exceptions. The compiler will not check these exceptions at compile time. In simple words, if a program throws an unchecked exception, and even if we didn't handle or declare it, the program would not give a compilation error. Usually, it occurs when the user provides bad data during the interaction with the program.

#### UncheckedExceptionExample1.java

```
class UncheckedExceptionExample1 {  
    public static void main(String args[])  
    {  
        int positive = 35;  
        int zero = 0;  
        int result = positive/zero;  
        //Give Unchecked Exception here.  
        System.out.println(result);  
    }  
}
```

In the above program, we have divided 35 by 0. The code would be compiled successfully, but it will throw an `ArithmeticException` error at runtime. On dividing a number by 0 throws the divide by zero exception that is a unchecked exception.

#### Output:

Exception in thread "main" [java.lang.ArithmeticException: / by zero](#)  
at uncheckedException.main([uncheckedException.java:7](#))

### Java Custom Exception

In Java, we can create our own exceptions that are derived classes of the `Exception` class. Creating our own Exception is known as custom exception or user-defined exception. Basically, Java custom exceptions are used to customize the exception according to user need.

Consider the example 1 in which `InvalidAgeException` class extends the `Exception` class. Using

the custom exception, we can have your own exception and message. Here, we have passed a string to the constructor of superclass i.e. `Exception` class that can be obtained using `getMessage()` method on the object we have created.

In this section, we will learn how custom exceptions are implemented and used in Java programs.

## Why use custom exceptions?

Java exceptions cover almost all the general type of exceptions that may occur in the programming. However, we sometimes need to create custom exceptions.

Following are few of the reasons to use custom exceptions:

- To catch and provide specific treatment to a subset of existing Java exceptions.
- Business logic exceptions: These are the exceptions related to business logic and workflow. It is useful for the application users or the developers to understand the exact problem.

In order to create custom exception, we need to extend Exception class that belongs to java.lang package.

Consider the following example, where we create a custom exception named WrongFileNameException:

```
public class WrongFileNameException extends Exception {  
    public WrongFileNameException(String errorMessage) {  
        super(errorMessage);  
    }  
}
```

### Example 1:

Let's see a simple example of Java custom exception. In the following code, constructor of InvalidAgeException takes a string as an argument. This string is passed to constructor of parent class Exception using the super() method. Also the constructor of Exception class can be called without using a parameter and calling super() method is not mandatory.

### TestCustomException1.java

```
// class representing custom exception  
class InvalidAgeException extends Exception  
1. {  
    public InvalidAgeException (String str)  
    {  
        // calling the constructor of parent Exception  
        super(str);  
    }  
}  
  
// class that uses custom exception InvalidAgeException  
public class TestCustomException1  
{
```

```

// method to check the age
static void validate (int age) throws InvalidAgeException{
    if(age < 18){

        // throw an object of user defined exception
        throw new InvalidAgeException("age is not valid to vote");
    }
    else {
        System.out.println("welcome to vote");
    }
}

// main method
public static void main(String args[])
{
    try

        // calling the method
        validate(13);

    catch (InvalidAgeException ex)

        System.out.println("Caught the exception");

        // printing the message from InvalidAgeException object
        System.out.println("Exception occurred: " + ex);

        System.out.println("rest of the code...");

}

```

### Output:-

Caught the exception  
 Exception occurred: [InvalidAgeException](#): age is not valid to vote rest  
 of the code...

### Java Multi-catch block

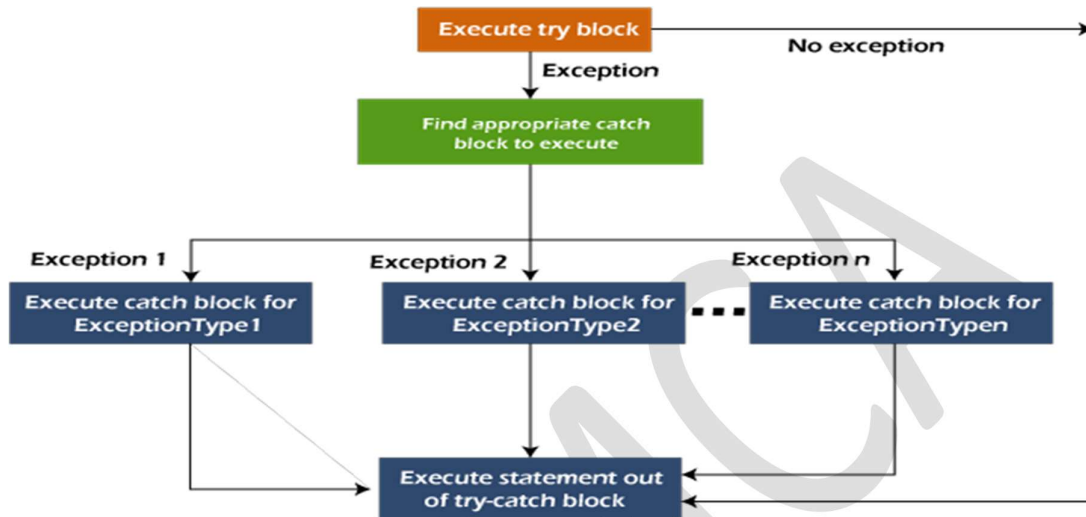
A try block can be followed by one or more catch blocks. Each catch block must contain a different exception handler. So, if you have to perform different tasks at the occurrence of different exceptions, use java multi-catch block.

### **Points to remember**

- At a time only one exception occurs and at a time only one catch block is executed.

- All catch blocks must be ordered from most specific to most general, i.e. catch for `ArithmeticException` must come before catch for `Exception`.

#### Flowchart of Multi-catch Block



#### Example 1

Let's see a simple example of java multi-catch block.

##### MultipleCatchBlock1.java

```

public class MultipleCatchBlock1 {
    public static void main(String[] args) {
        try {
            int a[] = new int[5];
            a[5] = 30/0;
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic Exception occurs");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("ArrayIndexOutOfBoundsException Exception occurs");
        } catch (Exception e) {
            System.out.println("Parent Exception occurs");
        }
    }
}

```

```

    }
    System.out.println("rest of the code");
}
}

```

### **Output:-**

Arithmetic Exception occurs rest  
of the code

### **Java Nested try block**

In Java, using a try block inside another try block is permitted. It is called as nested try block. Every statement that we enter a statement in try block, context of that exception is pushed onto the stack.

For example,  
the **inner try block** can be used to handle **ArrayIndexOutOfBoundsException** while the **outer try block** can handle the **ArithmeticException** (division by zero).

### **Why use nested try block**

Sometimes a situation may arise where a part of a block may cause one error and the entire block itself may cause another error. In such cases, exception handlers have to be nested.

```

public class NestedTryBlock {
    public static void main(String args[]) {
        //outer try block
        try {
            //inner try block 1
            try {
                System.out.println("going to divide by 0");
                int b = 39/0;
            }
            //catch block of inner try block 1
            catch (ArithmeticException e)
            {
                System.out.println(e);
            }
            //inner try block 2
            try {
                int a[] = new int[5];
                //assigning the value out of array bounds
                a[5] = 4;
            }
            //catch block of inner try block 2

```

```

    catch(ArrayIndexOutOfBoundsException e)
    {
        System.out.println(e);
    }
    System.out.println("other statement");
}
//catch block of outer try block
catch(Exception e)
{
    System.out.println("handled the exception (outer catch)");
}
System.out.println("normal flow..");
}
}

```

**output**

going to divide by 0

[java.lang.ArithmeticException: / by zero](#)

[java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 5](#)

other statement

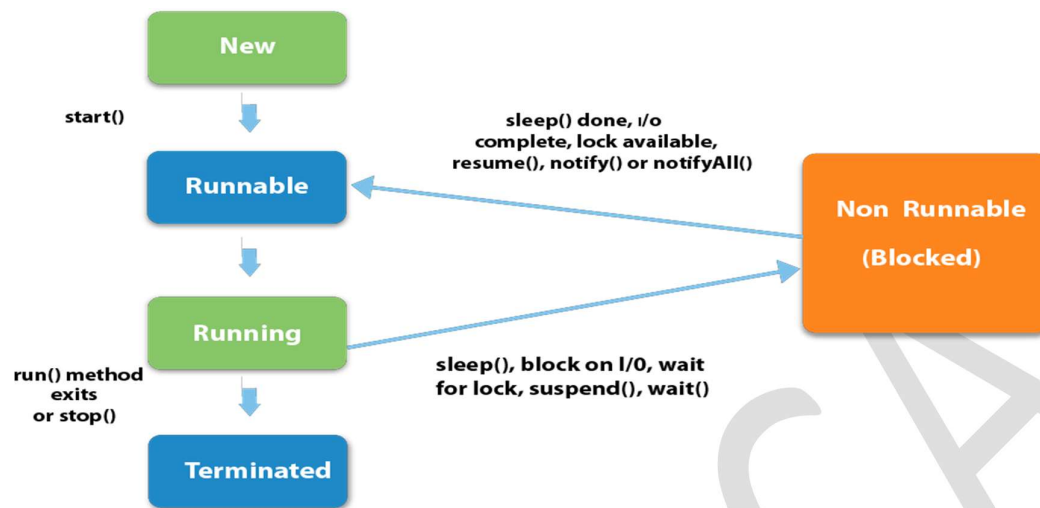
normal flow..

## Multithreading

### 1) What are the states in the lifecycle of a Thread?

A thread can have one of the following states during its lifetime:

1. **New:** In this state, a Thread class object is created using a new operator, but the thread is not alive. Thread doesn't start until we call the start() method.
2. **Runnable:** In this state, the thread is ready to run after calling the start() method. However, the thread is not yet selected by the thread scheduler.
3. **Running:** In this state, the thread scheduler picks the thread from the ready state, and the thread is running.
4. **Waiting/Blocked:** In this state, a thread is not running but still alive, or it is waiting for the other thread to finish.
5. **Dead/Terminated:** A thread is in terminated or dead state when the run() method exits.



## 2) Write the difference between notify() and notifyAll() in Java.

Both notify and notifyAll are the methods of thread class and used to provide notification for the thread. But there are some significant differences between both of these methods which we would discuss below.

Following are the important differences between notify and notifyAll.

| Sr. No. | Key                   | notify                                                                                                                                                                           | notifyAll                                                                                                                                 |
|---------|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 1       | Notification          | In case of multiThreading notify() method sends the notification to only one thread among the multiple waiting threads which are waiting for lock.                               | While notifyAll() methods in the same context sends the notification to all waiting threads instead of single one thread.                 |
| 2       | Thread identification | As in case of notify the notification is sent to single thread among the multiple waiting threads so it is sure that which of those waiting thread is going to receive the lock. | On other hand notifyAll sends notification to all waiting threads hence it is not clear which of the thread is going to receive the lock. |

| Sr. No. | Key             | notify                                                                                                                                                                                         | notifyAll                                                                                                                                                                                                   |
|---------|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3       | Risk factor     | In case of notify() method the risk of thread missing is high as notification is sent only single thread and if it misses that than no other thread would get notification and hence the lock. | While in case of notifyAll as notification is to all the waiting threads and hence if any thread misses the notification, there are other threads to do the job. Hence risk is less.                        |
| 4       | Performance     | Memory and CPU drain is less as compare to notifyAll as notification is sent to single one thread so performance is better as compare to notifyAll.                                            | On other hand as the cost of no notification is dropped and notification is sent to all waiting threads the memory and CPU drain is more as compare to notify and hence performance of notifyAll is lesser. |
| 5       | Interchangeable | In case of the notify() method as only single one thread is in picture hence no concept of thread Interchangeable is possible.                                                                 | While we should go for notifyAll() if all your waiting threads are interchangeable (the order they wake up doesn't matter).                                                                                 |

### 3) What are the advantages of multithreading?

Multithreading programming has the following advantages:

- Multithreading allows an application/program to be always reactive for input, even already running with some background tasks
- Multithreading allows the faster execution of tasks, as threads execute independently.
- Multithreading provides better utilization of cache memory as threads share the common memory resources.
- Multithreading reduces the number of the required server as one server can execute multiple threads at a time.

#### 4) What do you understand by inter-thread communication?

- The process of communication between synchronized threads is termed as inter-thread communication.
- Inter-thread communication is used to avoid thread polling in Java.
- The thread is paused running in its critical section, and another thread is allowed to enter (or lock) in the same critical section to be executed.
- It can be obtained by wait(), notify(), and notifyAll() methods.

#### 5) write an importance of wait(), notify() and notifyAll() methods in Java with its proper syntax?

The threads can communicate with each other through **wait()**, **notify()** and **notifyAll()** methods in Java. These are **final** methods defined in the **Object** class and can be called only from within a **synchronized** context. The **wait()** method causes the current thread to wait until another thread invokes the **notify()** or **notifyAll()** methods for that object. The **notify()** method **wakes up a single thread** that is waiting on that object's monitor. The **notifyAll()** method **wakes up all threads** that are waiting on that object's monitor. A thread waits on an object's monitor by calling one of the **wait()** method. These methods can throw **IllegalMonitorStateException** if the current thread is not the owner of the object's monitor.

##### wait() method Syntax

```
public final void wait() throws InterruptedException
```

##### notify() Method Syntax

```
public final void notify()
```

##### NotifyAll() method Syntax

```
public final void notifyAll()
```

#### 6) Differentiate between the Thread class and Runnable interface for creating a Thread?

The Thread can be created by using two ways.

- By extending the Thread class
- By implementing the Runnable interface

However, the primary differences between both the ways are given below:

- By extending the Thread class, we cannot extend any other class, as Java does not allow multiple inheritances while implementing the Runnable interface; we can also extend other base class(if required).
- By extending the Thread class, each of thread creates the unique object and associates with it while implementing the Runnable interface; multiple threads share the same object
- Thread class provides various inbuilt methods such as getPriority(), isAlive and many more while the Runnable interface provides a single method, i.e., run().

### 7) What is the difference between wait() and sleep() method?

| wait()                                           | sleep()                                        |
|--------------------------------------------------|------------------------------------------------|
| 1) The wait() method is defined in Object class. | The sleep() method is defined in Thread class. |
| 2) The wait() method releases the lock.          | The sleep() method doesn't release the lock.   |

### 8) What is the synchronization?

Synchronization is the capability to control the access of multiple threads to any shared resource. It is used:

1. To prevent thread interference.
2. To prevent consistency problem.

When the multiple threads try to do the same task, there is a possibility of an erroneous result, hence to remove this issue, Java uses the process of synchronization which allows only one thread to be executed at a time. Synchronization can be achieved in three ways:

- by the synchronized method
- by synchronized block
- by static synchronization

### 9) How to Achieve Multiple Inheritance in Java

Multiple Inheritance is a concept in object-oriented programming that allows a class to inherit properties from one or multiple parent classes. The problem arises when methods with similar

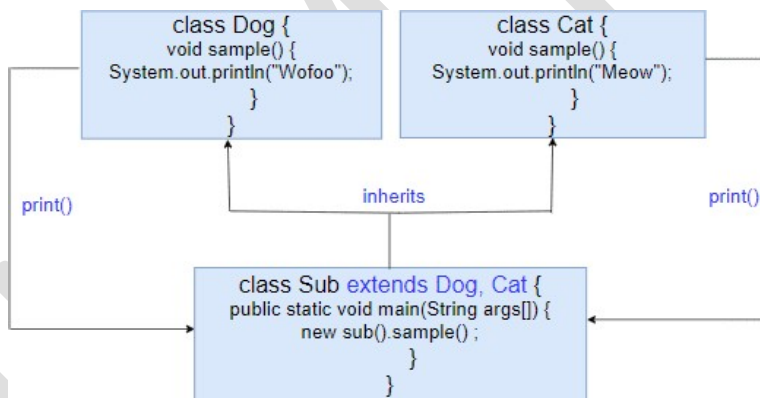
signatures exist in both subclasses and superclasses. The compiler is unable to determine which class method should be called first or given priority when calling the method.

Multiple inheritance creates ambiguity. For example, assume there is a Sub class and two classes named Cat and Dog, each with a method called sample( ). And if the class Sub inherits both **Cat** and **Dog**, there will be two copies of the sampling method, one from each superclass, making it impossible to decide which method to use.

**In Java, we can achieve multiple inheritance through the concept of interface.** An interface is like a class that has variables and methods, however, unlike a class, the methods in an interface are abstract by default. Multiple inheritance through interface occurs in Java when a class implements multiple interfaces or when an interface extends multiple interfaces.

In this example, we'll see how a Java program illustrates multiple inheritance via an interface.

- Each interface, Dog and Cat, has one abstract method, i.e., bark() and meow(), respectively.  
The Animal class implements the interfaces Dog and Cat.
- The main() method of the Demo class creates an object of the Animal class. Then, the bark() and meow() functions are called.
- Implementation of Multiple inheritance in Java through Interface



```
interface Dog
{ void bark();
}
```

```
interface Cat
{ void meow();}
```

```
class Animal implements Dog, Cat
{ public void bark()
{ System.out.println("Dog is barking");}
```

```
public void meow()

{ System.out.println("Cat is meowing");

}}

class Main {

public static void main(String args[])

{ Animal a = new Animal();

a.bark();

a.meow();

}

}
```

Output of the above code:

Dog is barking

Cat is meowing